

PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility

My repo is at <https://github.com/dlozan62/xv6-riscv/tree/riscv>.

```
xv6 kernel is booting  
  
init: starting sh  
$ uptime  
78 ticks  
$QEMU: Terminated
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

```
$ time1  
Usage: time1 command [args...]  
$ time1 ls  
.  
..  
README  
cat  
echo  
forktest  
grep  
init  
kill  
ln  
ls  
mkdir  
rm  
sh  
stressfs  
usertests  
grind  
wc  
zombie  
uptime  
time1  
logstress  
forphan  
dorphan  
sync  
console  
Time: 1 ticks elapsed time: 1 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

I changed three files in the operating system: proc.h, proc.c, and trap.c. For proc.h, I added the cputime field to the struct proc. This was to keep track of the CPU time. In proc.c, I initialized the cputime to 0 for the allocproc() function. This helps it start with zero CPU time every time. In the trap.c, I changed both usertrap and kerneltrap(). When a timer interrupt happens the process uses the time slice, I made it so that the cpu time is increment by one before the process yields the CPU. This is what keeps track of the ticks. I did it this way because it can be interrupted while running in either user or kernel mode.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

I made a new header file kernel/pstat.h for defining the rusage structure and it has the cputime as a field. I changed the user.h in the user because I needed to add the rusage structure and the wait2() function. I changed user/usys.pl and added wait2 as an entry. This is what will let the user program use the system call of wait2. I changed syscall.h to give a number for the system call. I then changed the syscall.c to add wait2 to the list of system-call functions. For sysproc.c, I added the sys_wait2() function so that it gets the arguments passed from user program and then it calls the kernel level wait2() function. For proc.c, I added the kernel implementation of wait2() and based it off the kwait() function. I also made it copy the child's CPU time into the rusage. The last file I changed was the defs.h so that I could add the wait2() function from the kernel so other kernel files could use it.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

```
init: starting sh
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul
Time: 4 ticks
elapsed time: 4 ticks, cpu time: 4 ticks, 100% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

I learned how system calls work in xv6 and how the user programs communicate with the kernel. I learned implementing uptime and creating the different time commands, also how to track the CPU time for the processes. The most important to me which I learned was how to change multiple parts of the xv6 like the system-call tables, Makefile, and user programs. A difficulty I face was making sure each of the files were all connected properly. I had some errors because of syntax, and I misspelled some variables but I was able to correct them all. I also had to figure out how to correctly get my branch and push it to my fork. I overcame these by looking all over my code, rebuilding the xv6 and testing the programs. I did also use some AI to help me with spelling mistakes or errors such as those.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).